

Clase Unidad 1

Representación y Organización de Datos

Informática II - Universidad Nacional de
La Rioja

Autor: Marcelo Martinez

Índice

1. Introducción

2. Bits y Bytes

3. Sistemas Numéricos

4. Representación de Datos Primitivos

5. Estructuras de Datos en Memoria

6. Memoria

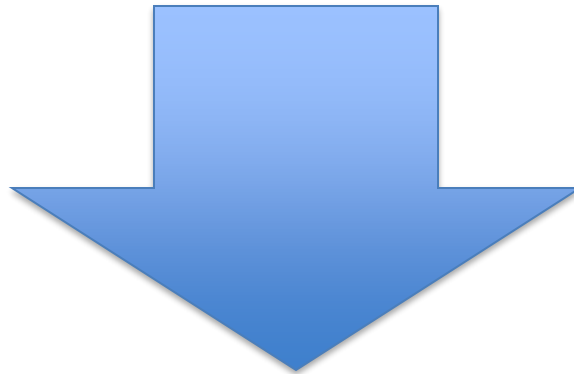
7. Almacenamiento y Acceso

8. Conceptos Avanzados

9. Conclusión

Introducción

- Exploraremos los conceptos fundamentales sobre la representación y organización de datos en un sistema informático, incluyendo bits, bytes, sistemas numéricos, estructuras de datos, y la jerarquía de memoria.



Bits y Bytes

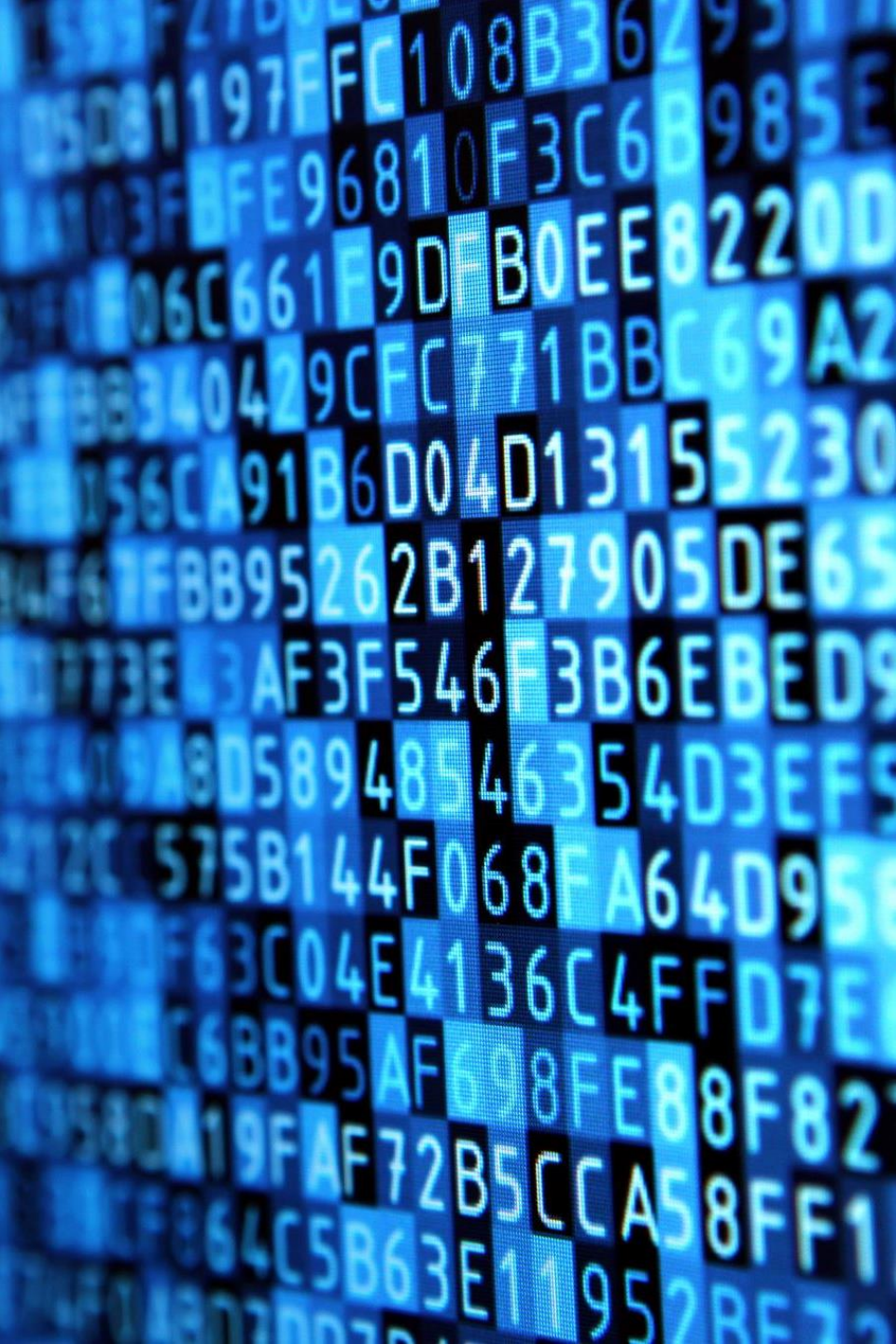
Bits:

- Definición: Unidad mínima de información, puede ser 0 o 1.
- Uso: Representación de datos y realización de operaciones.

Bytes:

- Definición: Compuesto por 8 bits.
- Uso: Representa 256 valores diferentes, suficiente para almacenar un carácter.





Sistemas Numéricos

- Decimal (Base 10):
 - - Ejemplo: 345 en decimal.
- Binario (Base 2):
 - - Ejemplo: 1011 en binario.
- Hexadecimal (Base 16):
 - - Ejemplo: 2F en hexadecimal.

TABLA ASCII - Decimal

[illegible]

TABLA ASCII - BINARIO

CODIFICACIÓN BINARIA ASCII

Character	Binary Code	Character	Binary Code	Character	Binary Code	Character	Binary Code	Character	Binary Code
A	01000001	Q	01010001	g	01100111	w	01110111	-	00101101
B	01000010	R	01010010	h	01101000	x	01111000	.	00101110
C	01000011	S	01010011	i	01101001	y	01111001	/	00101111
D	01000100	T	01010100	j	01101010	z	01111010	0	00110000
E	01000101	U	01010101	k	01101011	!	00100001	1	00110001
F	01000110	V	01010110	l	01101100	"	00100010	2	00110010
G	01000111	W	01010111	m	01101101	#	00100011	3	00110011
H	01001000	X	01011000	n	01101110	\$	00100100	4	00110100
I	01001001	Y	01011001	o	01101111	%	00100101	5	00110101
J	01001010	Z	01011010	p	01110000	&	00100110	6	00110110
K	01001011	a	01100001	q	01110001	'	00100111	7	00110111
L	01001100	b	01100010	r	01110010	(	00101000	8	00111000
M	01001101	c	01100011	s	01110011	)	00101001	9	00111001
N	01001110	d	01100100	t	01110100	*	00101010	?	00111111
O	01001111	e	01100101	u	01110101	+	00101011	@	01000000
P	01010000	f	01100110	v	01110110	,	00101100	_	01011111

TABLA ASCII - HEXADECIMAL

DEC....HEX....ASCII	DEC....HEX....ASCII	DEC....HEX....ASCII	DEC....HEX....ASCII
0 00 NUL	32 20	64 40 @	96 60 '
1 01 SOH	33 21 !	65 41 A	97 61 a
2 02 STX	34 22 "	66 42 B	98 62 b
3 03 ETX	35 23 #	67 43 C	99 63 c
4 04 EOT	36 24 \$	68 44 D	100 64 d
5 05 ENQ	37 25 %	69 45 E	101 65 e
6 06 ACK	38 26 &	70 46 F	102 66 f
7 07 BEL	39 27 '	71 47 G	103 67 g
8 08 BS	40 28 (	72 48 H	104 68 h
9 09 HT	41 29)	73 49 I	105 69 i
10 0A LF	42 2A *	74 4A J	106 6A j
11 0B VT	43 2B +	75 4B K	107 6B k
12 0C FF	44 2C ,	76 4C L	108 6C l
13 0D CR	45 2D -	77 4D M	109 6D m
14 0E SO	46 2E	78 4E N	110 6E n
15 0F SI	47 2F /	79 4F O	111 6F o
16 10 DLE	48 30 0	80 50 P	112 70 p
17 11 DC1	49 31 1	81 51 Q	113 71 q
18 12 DC2	50 32 2	82 52 R	114 72 r
19 13 DC3	51 33 3	83 53 S	115 73 s
20 14 DC4	52 34 4	84 54 T	116 74 t
21 15 NAK	53 35 5	85 55 U	117 75 u
22 16 SYN	54 36 6	86 56 V	118 76 v
23 17 ETB	55 37 7	87 57 W	119 77 w
24 18 CAN	56 38 8	88 58 X	120 78 x
25 19 EM	57 39 9	89 59 Y	121 79 y
26 1A SUB	58 3A :	90 5A Z	122 7A z
27 1B ESC	59 3B ;	91 5B [	123 7B {
28 1C FS	60 3C <	92 5C \	124 7C
29 1D GS	61 3D =	93 5D]	125 7D }
30 1E RS	62 3E >	94 5E ^	126 7E ~
31 1F US	63 3F ?	95 5F _	127 7F

TABLA ASCII – De – Bi - He

Construir una tabla que
tenga todas las
representaciones. (TAREA
PARA USTEDES)



Conversiones entre sistemas numéricos – Decimal a binario

Conversión de Decimal a Binario

Paso 1: Dividir el Número Decimal

- Divide el número decimal entre 2.
- Anota el cociente y el residuo.

Ejemplo:

Número Decimal = 18

División	Cociente	Residuo
$18 \div 2$	9	0
$9 \div 2$	4	1
$4 \div 2$	2	0
$2 \div 2$	1	0
$1 \div 2$	0	1

Conversiones entre sistemas numéricos – Decimal a binario

Paso 2: Leer los Residuos de Abajo hacia Arriba

- El número binario se forma leyendo los residuos desde el último obtenido hasta el primero.

Residuos:

1
0
0
1
0

Conversiones entre sistemas numéricos – Decimal a binario

Paso 3: Formar el Número Binario

- Los residuos leídos de abajo hacia arriba forman el número binario.

$$18_{\text{decimal}} = 10010_{\text{binario}}$$

Ejemplo Adicional

Número Decimal = 43

División	Cociente	Residuo
$43 \div 2$	21	1
$21 \div 2$	10	1
$10 \div 2$	5	0
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1

$$43_{\text{decimal}} = 101011_{\text{binario}}$$

Conversiones entre sistemas numéricos – Decimal a binario

Resumen del Proceso

1. Divide el número decimal entre 2.
2. Anota el cociente y el residuo.
3. Repite el proceso con el cociente hasta que sea 0.
4. Lee los residuos de abajo hacia arriba para formar el número binario.

Conversiones entre sistemas numéricos - Binario a decimal

Paso 1: Entender la Posición de Cada Dígito Binario

- Cada posición en un número binario representa una potencia de 2, comenzando desde la derecha con 2^0 .

Ejemplo:

Número Binario = 10110

Posiciones:

2^4	2^3	2^2	2^1	2^0
1	0	1	1	0

Conversiones entre sistemas numéricos - Binario a decimal

Paso 2: Multiplicar Cada Dígito Binario por su Potencia Correspondiente

- Multiplica cada dígito del número binario por 2 elevado a la potencia de su posición, comenzando desde la derecha.

Dígito	Posición (Potencia de 2)	Cálculo
1	2^4	$1 \times 2^4 = 16$
0	2^3	$0 \times 2^3 = 0$
1	2^2	$1 \times 2^2 = 4$
1	2^1	$1 \times 2^1 = 2$
0	2^0	$0 \times 2^0 = 0$

Conversiones entre sistemas numéricos - Binario a decimal

Paso 3: Sumar los Resultados de las Multiplicaciones

- Suma todos los valores obtenidos en el paso anterior.

$$16 + 0 + 4 + 2 + 0 = 22$$

$$10110_{\text{binario}} = 22_{\text{decimal}}$$

Conversiones entre sistemas numéricos - Binario a decimal

Ejemplo Adicional

Número Binario = 1101

Posiciones:

$$\begin{array}{c|c|c|c} 2^3 & 2^2 & 2^1 & 2^0 \\ \hline 1 & 1 & 0 & 1 \end{array}$$

Dígito	Posición (Potencia de 2)	Cálculo
1	2^3	$1 \times 2^3 = 8$
1	2^2	$1 \times 2^2 = 4$
0	2^1	$0 \times 2^1 = 0$
1	2^0	$1 \times 2^0 = 1$

$$8 + 4 + 0 + 1 = 13$$

$$1101_{\text{binario}} = 13_{\text{decimal}}$$

Conversiones entre sistemas numéricos - Binario a decimal

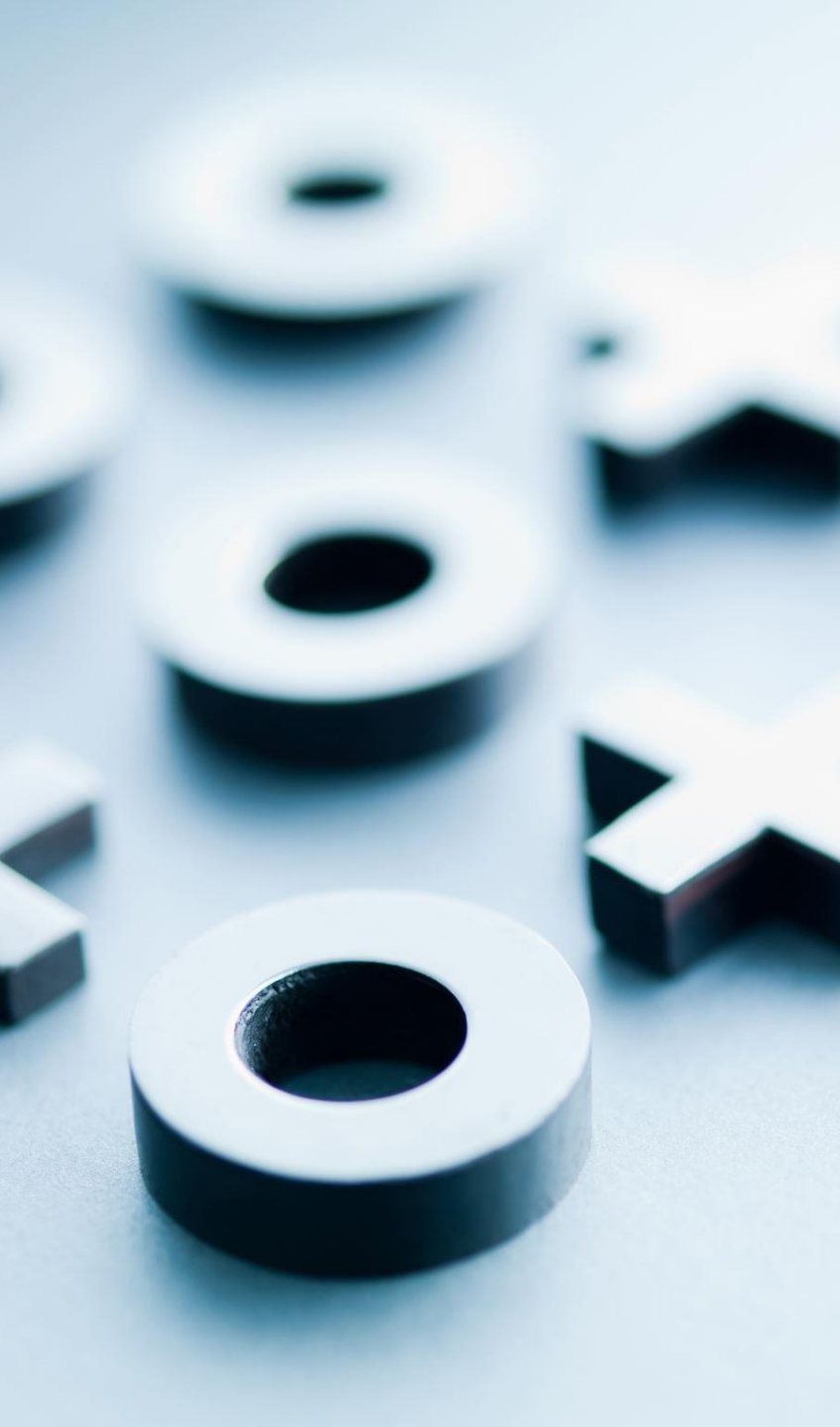
Resumen del Proceso

1. Identifica la posición de cada dígito binario, empezando de derecha a izquierda.
2. Multiplica cada dígito binario por 2 elevado a la potencia de su posición.
3. Suma todos los resultados para obtener el número decimal.

Representación de Datos Primitivos

Enteros:

- - Signo y Magnitud
 - En este método, el bit más significativo se usa para representar el signo del número. Si el bit es 0, el número es positivo; si es 1, el número es negativo. Los bits restantes representan la magnitud del número.
 - **Ejemplo:** Para un número de 4 bits, 0101 representa +5, mientras que 1101 representa -5.
- - Complemento a Dos
 - Es el método más común para representar números negativos en binario. Para obtener el complemento a dos de un número, se invierten todos los bits (complemento a uno) y luego se suma 1.
 - **Ejemplo:** Para representar -5 en complemento a dos con 4 bits, se toma 0101 (que es +5), se invierten los bits obteniendo 1010, y luego se suma 1, resultando en 1011.



Representación de Datos Primitivos

Flotantes:

- IEEE 754
 - Es el estándar para la representación de números en coma flotante. Utiliza una estructura con tres partes: el bit de signo, el exponente y la mantisa (o fracción).
 - **Ejemplo:** Para un número de precisión simple (32 bits), se tiene 1 bit para el signo, 8 bits para el exponente y 23 bits para la mantisa.



Representación de Datos Primitivos

Caracteres:

- **ASCII:**
 - **Definición:** Código estándar para el intercambio de información. Usa 8 bits para representar caracteres, permitiendo hasta 256 caracteres diferentes.
 - **Ejemplo:** La letra 'A' se representa como 65 en decimal o 01000001 en binario.
- **Unicode:**
 - **Definición:** Soporta una amplia gama de caracteres y símbolos de diferentes lenguajes y sistemas de escritura. Utiliza más bits que ASCII, comúnmente 16 bits.
 - **Ejemplo:** El carácter 'A' en Unicode sigue siendo 65, pero puede representar caracteres como '漢' (hanzi) con códigos mucho más altos.

¿Qué es la memoria en un Sistema de Computación?

La memoria es un componente esencial del sistema que se encarga de almacenar datos y programas de manera que el procesador/CPU pueda acceder a ellos de forma rápida y eficiente. Es crucial para el almacenamiento y la gestión de datos, permitiendo al procesador realizar cálculos, ejecutar programas y gestionar la información de manera eficiente. Sin una memoria adecuada, el sistema no podría funcionar correctamente y su rendimiento se vería seriamente afectado.



La memoria en un sistema de computación tiene varios propósitos importantes

- **Almacenamiento de Datos Temporales:** La memoria permite almacenar datos temporales mientras se realizan operaciones. Esto incluye datos que el procesador necesita para ejecutar instrucciones en curso.
- **Ejecución de Programas:** Los programas y aplicaciones se cargan en la memoria para que el procesador pueda ejecutar sus instrucciones. Sin memoria, el procesador no podría acceder a los datos necesarios para ejecutar programas.
- **Rendimiento y Velocidad:** La memoria, especialmente la memoria RAM (Memoria de Acceso Aleatorio), proporciona un acceso rápido a los datos, lo que mejora significativamente el rendimiento del sistema. La memoria de mayor velocidad permite que el procesador acceda a los datos de manera más eficiente.
- **Persistencia de Datos:** Aunque la memoria RAM almacena datos temporalmente, los datos también se pueden guardar en almacenamiento persistente, como discos duros o unidades SSD. Esto asegura que los datos se conserven incluso cuando el sistema se apaga.



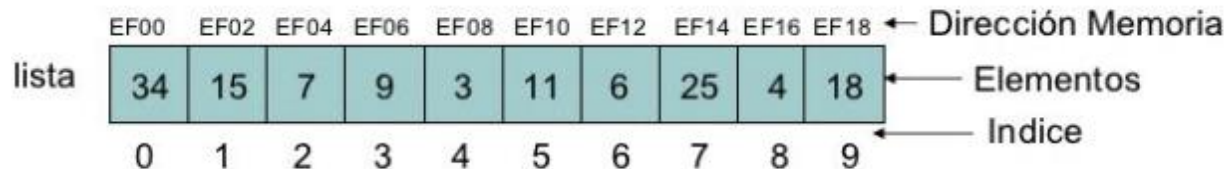
Estructuras de Datos en Memoria

Arreglos:

- Es una colección de elementos del mismo tipo, accesibles mediante índices.
- Ejemplo:** `int arr[5] = {1, 2, 3, 4, 5};` declara un arreglo de enteros con 5 elementos.

Índice:

Identifica la posición del elemento en el arreglo. También se le conoce como subíndice. Deben ser enteros: constantes, variables o expresiones. El primer valor comienza generalmente por 0.



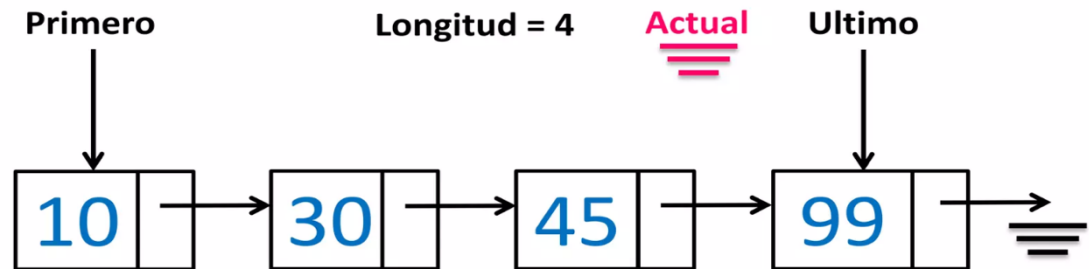
Los elementos del arreglo lista del ejemplo serían:

<code>lista[0] = 34</code>	primer elemento del arreglo
<code>lista[1] = 15</code>	segundo elemento del arreglo
.....	
<code>lista[9] = 18</code>	último elemento del arreglo

Estructuras de Datos en Memoria

Listas Enlazadas:

- Se entienden como un conjunto de nodos donde cada nodo contiene un dato y un puntero al siguiente nodo.
- Ejemplo:** Ideal para estructuras dinámicas donde la memoria puede ser gestionada eficientemente.

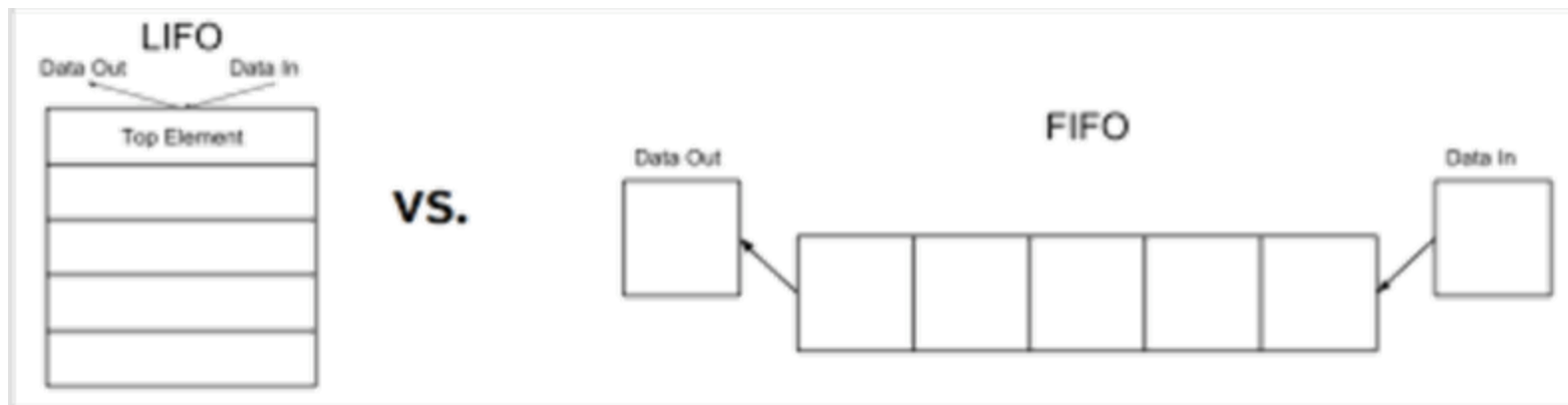


Nodo Simple

Estructuras de Datos en Memoria

Pilas y Colas:

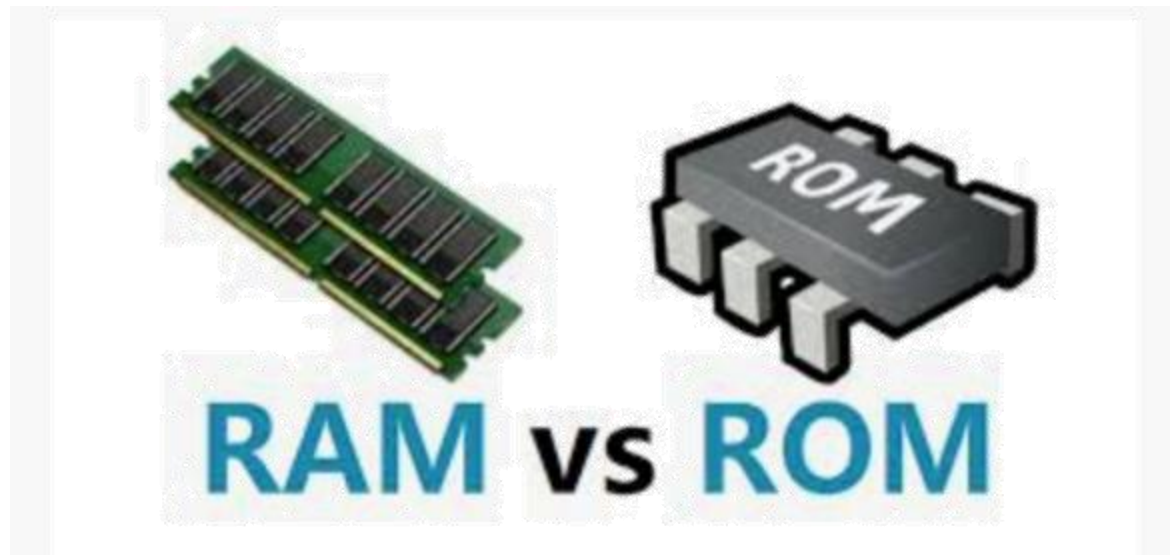
- **Pilas (LIFO):**
 - Last In, First Out. El último elemento en entrar es el primero en salir.
 - **Ejemplo:** Ideal para la gestión de llamadas a funciones.
- **Colas (FIFO):**
 - First In, First Out. El primer elemento en entrar es el primero en salir.
 - **Ejemplo:** Utilizado en sistemas de impresión y tareas en cola.



Memoria

Tipos de Memoria:

- **RAM (Memoria de Acceso Aleatorio):**
 - Volátil y de alta velocidad, usada como memoria principal del sistema.
- **ROM (Memoria de Solo Lectura):**
 - No volátil, contiene instrucciones permanentes.



Memoria

Jerarquía de Memoria:

- **Registro:**
 - Memoria más rápida y pequeña dentro del CPU.
- **Caché:**
 - Memoria intermedia entre RAM y registros, mejora la velocidad de acceso a datos frecuentes.
- **RAM:**
 - Memoria principal del sistema, volátil y de alta capacidad.
- **Disco Duro:**
 - Memoria de almacenamiento masivo y permanente.



Almacenamiento y Acceso

Sistemas de Archivos:

- **Definición:**
 - Método y estructura para almacenar y organizar archivos.
- **Ejemplos:**
 - NTFS, FAT32, ext4.



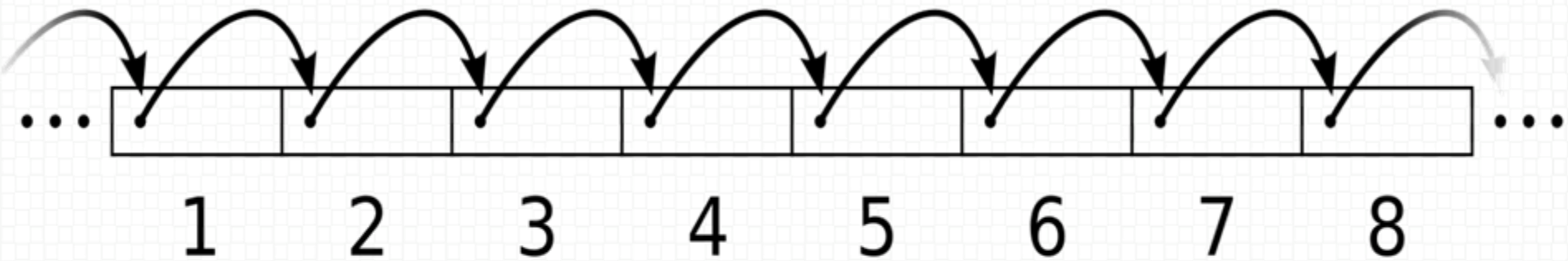
Almacenamiento y Acceso

Métodos de Acceso:

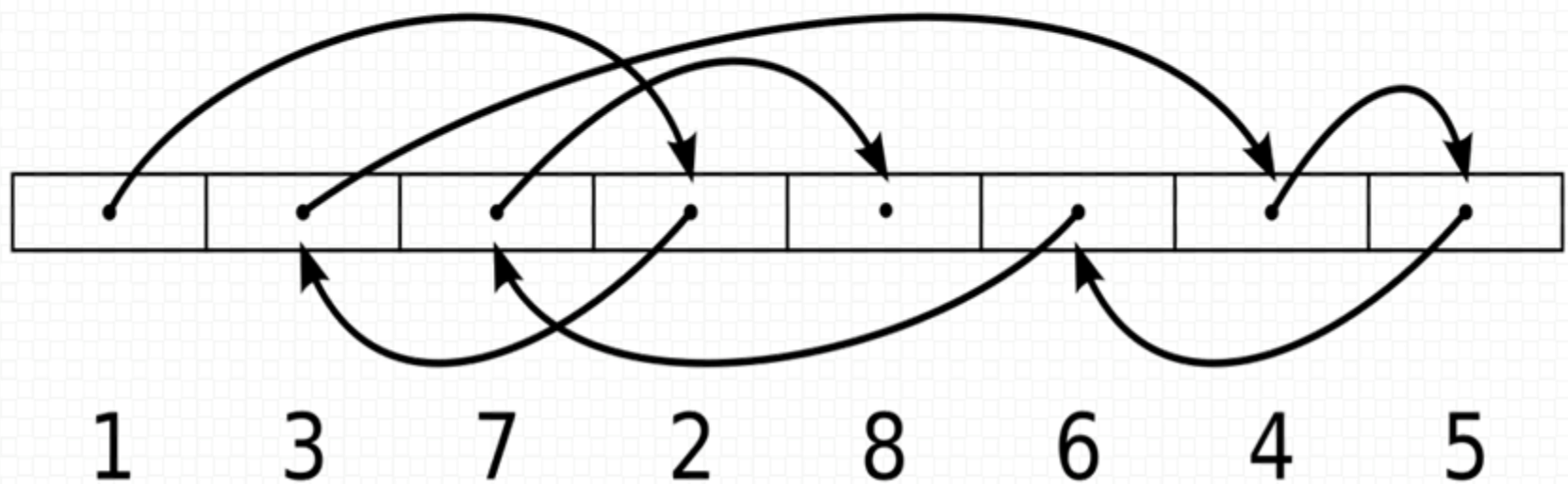
- **Secuencial:**
 - Lectura o escritura de datos en un orden específico.
- **Aleatorio:**
 - Acceso directo a cualquier punto del archivo sin recorrer datos anteriores.



Acceso secuencial



Acceso aleatorio



Conceptos Avanzados

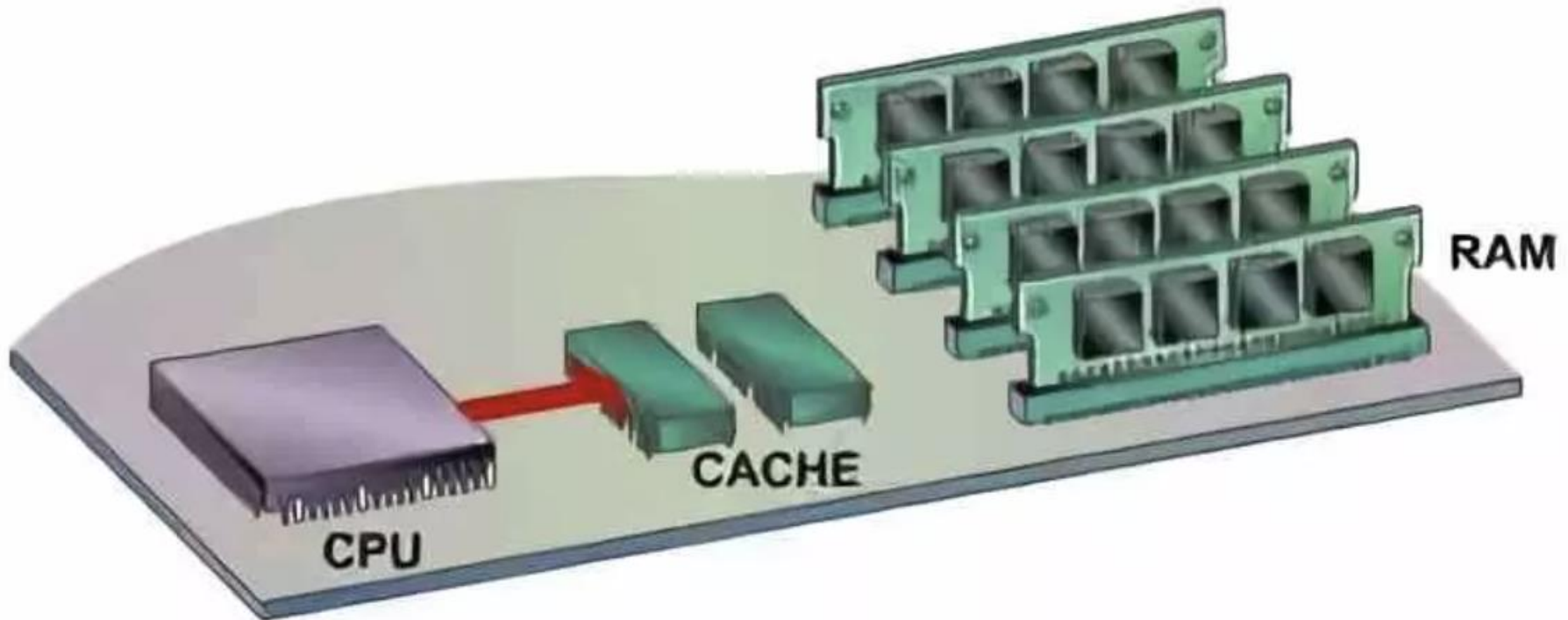
Virtualización de Memoria:

- Es una técnica que proporciona una capa de abstracción para la gestión de la memoria.
- **Se usa para:** Mejorar la eficiencia y seguridad en sistemas operativos.



Conceptos Avanzados

- **Definición:** Mecanismos para asegurar que todas las copias de datos en caché sean consistentes.
- **Protocolos:** MESI (Modified, Exclusive, Shared, Invalid).



Conceptos Avanzados

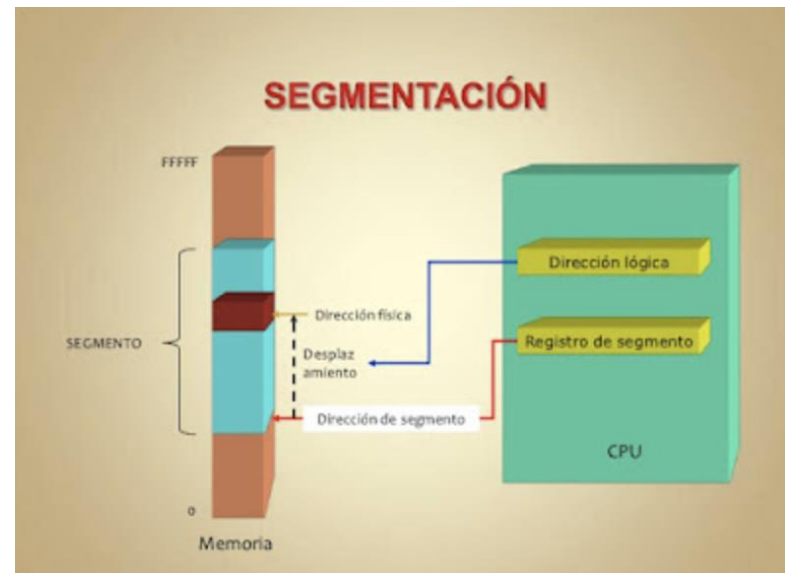
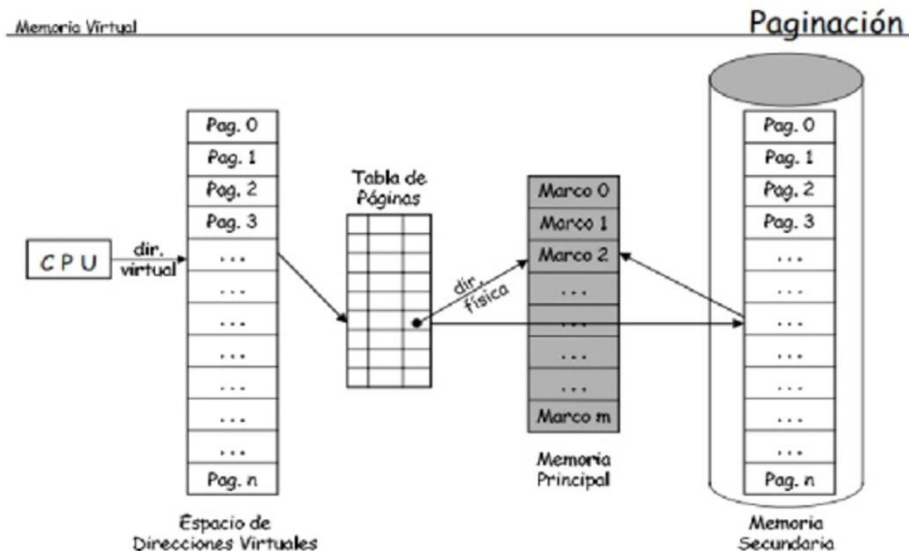
Políticas de Reemplazo de Caché:

- **LRU (Least Recently Used):** Reemplaza el dato menos recientemente utilizado.
- **FIFO (First In, First Out):** Reemplaza el dato más antiguo.

Conceptos Avanzados

Gestión de Memoria en Sistemas Operativos

- **Paginación:** Divide la memoria física en bloques de tamaño fijo.
- **Segmentación:** Divide la memoria en segmentos de tamaño variable.



Conclusión

Hemos recorrido los sistemas de numeración, Introducción, Bits y Bytes, la representación de Datos Primitivos, como se estructuran los datos memoria. Presentamos los conceptos de memoria, almacenamiento y acceso, para finalizar en conceptos más avanzados que se desarrollaran en otras cátedras en los siguientes años de la carrera.

Para qué?, sigamos con la siguiente unidad...

